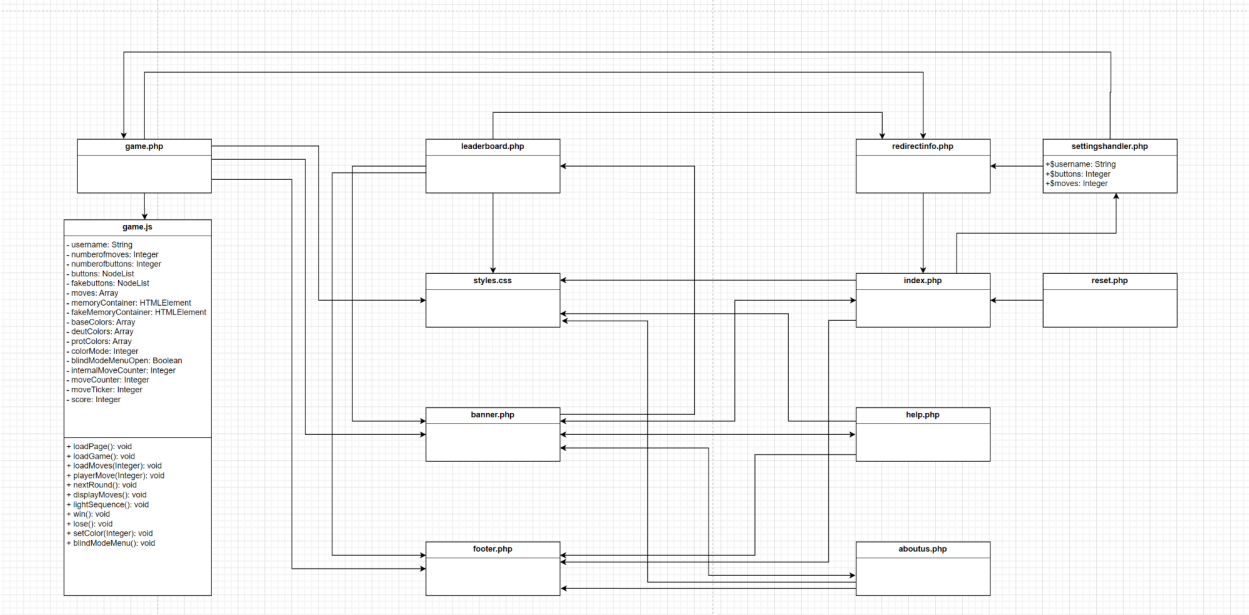
*(We **have** added these products to the system requirements page but decided it would be easier to submit a separate page for just these two points)*

1. The class diagrams should display all the primary classes, attributes, and methods. In addition, you must indicate the relationship between the classes (association, generalization, aggregation …etc.) with the multiplicity indicators.



2. Describe in detail the architecture of the system. Name the main parts of your system and list the design goals that led to the adoption of the proposed architecture. Also, describe the data that your system stores and the method of storage.

The architecture of our system is layered. Abstractly, the first layer consists of PHP and CSS files that define the content and the layout of the interface - the webpage. The second layer, utilizes PHP templates, specifically the "redirectifninfo.php" file, to handle user input for settings and identification. Since this site only stores data locally, no authentication is necessary, and user input can be redefined if it is outside the bounds of acceptable input. For example, one of our non-functional requirements is to limit buttons to between four and eight. If a user enters forty-two, we will default to eight. This layer also authorizes users to enter certain pages only when pertinent parameters have been defined. A user cannot enter game.php unless he has entered a username and appropriate settings on index.php and attempting to do so will redirect him to index.php.

The third layer is concerned with core application functions of the system. game.js implements the memory game and interacts with the webpage to display moves, update records, and display updated information. game.php submits the data to leaderboard.php through PHP sessions.

Since this application utilizes a local database, there is no communication with a server. In order to store data for the duration of the session, the leaderboard data records will be stored in the session superglobal and iteratively displayed on the leaderboard page.

Any data that needs to be handed from page to page will be stored in the session superglobal. If the data is no longer needed, it will be destroyed or overwritten. This architecture seemed optimal for the web application that we have developed due to the neat separation of functionalities into layers.